Normal MLE, OLS, and Censoring

Back to normal MLE

Maximizing the Likelihood: μ

$$\begin{aligned}\frac{\partial L[y|\hat{\mu},\hat{\sigma}]}{\partial \mu} &= \frac{1}{\sigma^2}\sum_{i=1}^{n}(y_i-\hat{\mu}) \\ 0 &= \sum_{i=1}^{n} y_i - n\hat{\mu} \\ \hat{\mu} &= \frac{1}{n}\sum_{i=1}^{n} y_i\end{aligned}$$

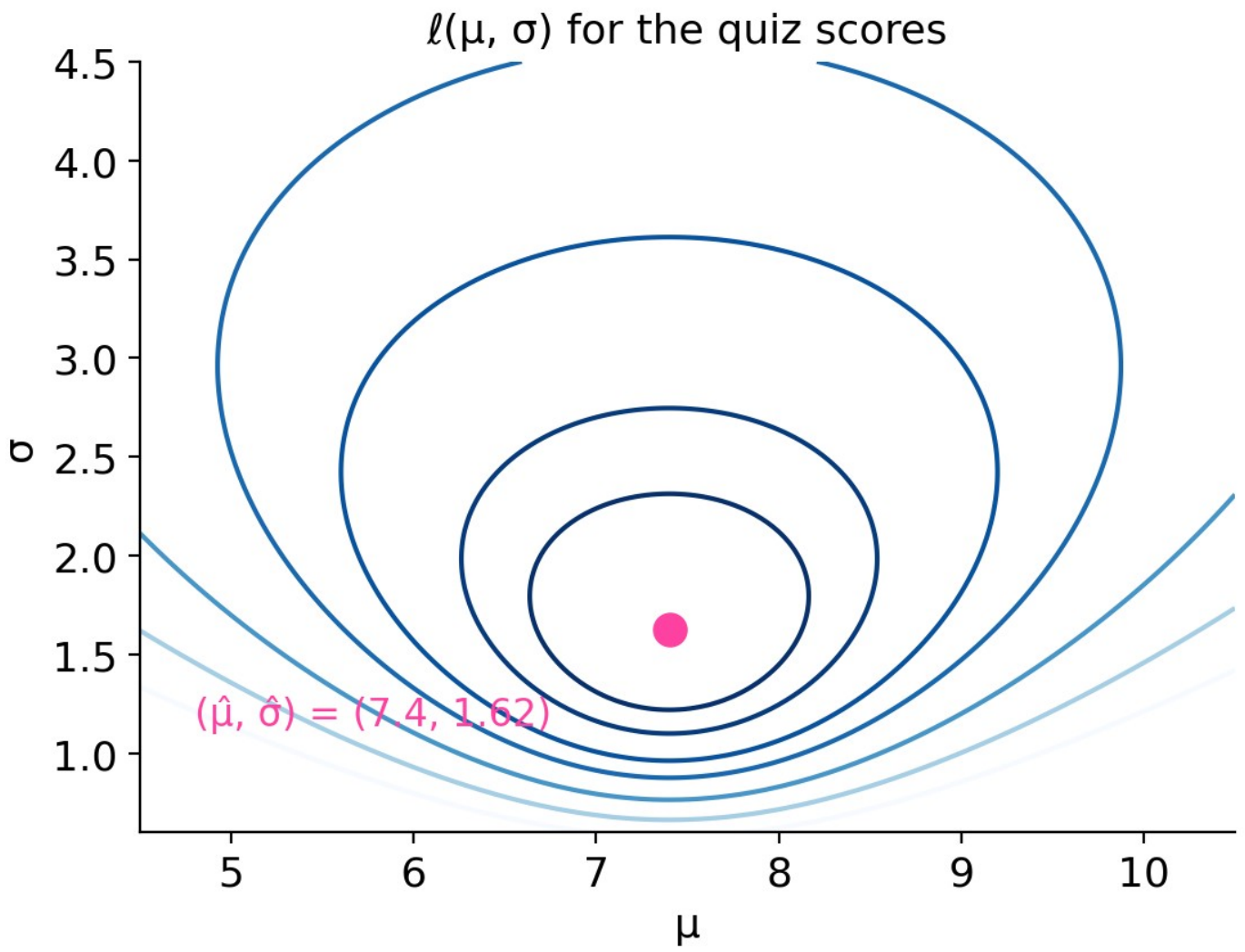
The sample mean!

Maximizing the Likelihood: σ

$$\begin{aligned}\frac{\partial L[y|\hat{\mu},\hat{\sigma}]}{\partial \sigma} &= \frac{1}{\hat{\sigma}^3}\sum_{i=1}^{n}(y_i-\hat{\mu})^2 - \frac{n}{\hat{\sigma}} \\ 0 &= \sum_{i=1}^{n}(y_i-\hat{\mu})^2 - n\hat{\sigma}^2 \\ \hat{\sigma}^2 &= \frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{\mu})^2\end{aligned}$$

The sample variance!

Both Parameters at Once



4. Where do models come from?

The mean-error decomposition

Where Do Models Come From?

- $X = \mu + \sigma\varepsilon$
  - model the error term explicitly (parametric)
  - link the model to data via parameters
  - MLE for estimate of parameters

Mean-Error Decomposition

- RV Y with mean μ and SE σ: y_i = μ + σε_i
- ε_i = shock, with mean 0, variance 1, density f(e)

$$\varepsilon_i = \frac{y_i - \mu}{\sigma}$$

- Rearrange and use the normal density

$$f(x_i \mid \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

$$\log\left(\prod_i f(\varepsilon_i)\right) = \sum_i \log\left(f\left(\frac{y_i - \mu}{\sigma}\right)\right)$$

$$= -n\log\sigma - \frac{n}{2}\log(2\pi) - \frac{1}{2}\sum_i e_i^2$$

- For a fixed σ, maximizing L over μ ≡ **least squares**

Regression Is the Same Idea

Similarly, in regression: $y_i = x_i^\top \beta + \sigma \varepsilon_i$

Model μ by a linear function: replace $\mu_i \to x_i^\top \beta$

How does this relate to regression?

$$y_i = x_i\beta + \sigma\varepsilon$$

- OLS is secretly MLE on normal RV
- We want to estimate betas – coefficients on X that allow us to predict Y: Y is assumed N(xb, sigma)
- Our expected value of y knowing x (i.e. predicted y) is really just x*B-hat (conditional expectation function)
- The normal RV is the error: we are assuming our data is normally distributed around the mean estimate: sigma * e, where e ~ N(0,1)

$$L(\beta, \sigma) = -n \log\left(\sqrt{2\pi}\right) - n \log(\sigma) - \frac{1}{2\sigma^2} \underbrace{\sum_{i=1}^{n} (y_i - x_i\beta)^2}_{\text{Sum of Squared Error}}$$

Here we know the solution (OLS) but in general Gradient ascent this thing to calculate it numerically
This is what statsmodels will do for us

Aside

- If we use something that is not Normal for the error, we will be maximizing a different likelihood == minimizing a different loss function == running a different kind of regression
- **Each loss function is connected probabilistically to a density function: You cannot separate these things conceptually**

Prediction intervals vs CI

- CI: Confidence in our estimate
  - This is the confidence in our mean, which needs the standard error

$$\left(\hat{\mu} - 1.96 \times \frac{\hat{\sigma}}{\sqrt{n}}, \quad \hat{\mu} + 1.96 \times \frac{\hat{\sigma}}{\sqrt{n}}\right)$$ (95% CI)

- PI: Confidence in a prediction y given x
  - This is our confidence in a prediction on a new x, which uses the standard deviation of the data we have

$$(\hat{\mu} - 1.96 \times \hat{\sigma}, \quad \hat{\mu} + 1.96 \times \hat{\sigma})$$ (95% PI for mean with Normal model)

$$\left(x\hat{\beta} - 1.645 \times \hat{\sigma}, \quad x\hat{\beta} + 1.645 \times \hat{\sigma}\right)$$ (90% PI for y from regression model)

Example: Ames House Prices

- Model: log(price) ~ log(area) + overall quality + full baths, fit by OLS on 80% of the Ames data (the rest is test data). OLS with normal errors is the MLE from earlier
- The fitted line predicts the log price. $R^2 = 0.767$ on the training data
- A **90% prediction interval** for one house is its predicted log price ± 1.645 $\hat{\sigma}$. It covers where a new house's price should fall
- It is not a confidence interval for the mean price, so it does not shrink as n grows: the house-to-house noise σ is always there
- Check: on the 20% test set, **92.3%** of the 90% intervals contain the true log price, close to the 90% target

Ames: The Fitted Model with CI in statsmodels

```python
import statsmodels.formula.api as smf

train, test = train_test_split(df, test_size=0.2, random_state=42)
# C() creates category indicators, with the lowest level as the reference.
# The log-area coefficient is the price elasticity, holding these categories fixed.
model = smf.ols(
    'np.log(price) ~ np.log(area) + C(Q("Overall.Qual")) + C(Q("Full.Bath"))',
    data=train,
).fit()
model.summary()
```

OLS Regression Results

Dep. Variable:	np.log(price)	R-squared:	0.767
Model:	OLS	Adj. R-squared:	0.765
Method:	Least Squares	F-statistic:	546.1
Date:	Tue, 10 Nov 2026	Prob (F-statistic):	0.00
Time:	01:33:52	Log-Likelihood:	519.94
No. Observations:	2344	AIC:	-1010.
Df Residuals:	2329	BIC:	-923.5
Df Model:	14		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	8.1191	0.161	50.465	0.000	7.804	8.435
C(Q("Overall.Qual"))[T.2]	0.2218	0.115	1.921	0.055	-0.005	0.448
C(Q("Overall.Qual"))[T.3]	0.6320	0.105	6.032	0.000	0.427	0.837
C(Q("Overall.Qual"))[T.4]	0.8305	0.100	8.334	0.000	0.635	1.026
C(Q("Overall.Qual"))[T.5]	1.0497	0.099	10.604	0.000	0.856	1.244
C(Q("Overall.Qual"))[T.6]	1.1540	0.099	11.633	0.000	0.959	1.349
C(Q("Overall.Qual"))[T.7]	1.3127	0.100	13.191	0.000	1.118	1.508
C(Q("Overall.Qual"))[T.8]	1.5317	0.100	15.310	0.000	1.335	1.728
C(Q("Overall.Qual"))[T.9]	1.7877	0.102	17.573	0.000	1.588	1.987
C(Q("Overall.Qual"))[T.10]	1.8210	0.109	16.762	0.000	1.608	2.034
C(Q("Full.Bath"))[T.1]	-0.1756	0.063	-2.790	0.005	-0.299	-0.052
C(Q("Full.Bath"))[T.2]	-0.1236	0.063	-1.961	0.050	-0.247	1.47e-05
C(Q("Full.Bath"))[T.3]	-0.0010	0.069	-0.015	0.988	-0.137	0.135
C(Q("Full.Bath"))[T.4]	-0.1112	0.129	-0.860	0.390	-0.365	0.142
np.log(area)	0.3925	0.018	22.046	0.000	0.358	0.427

Omnibus:	511.772	Durbin-Watson:	2.042

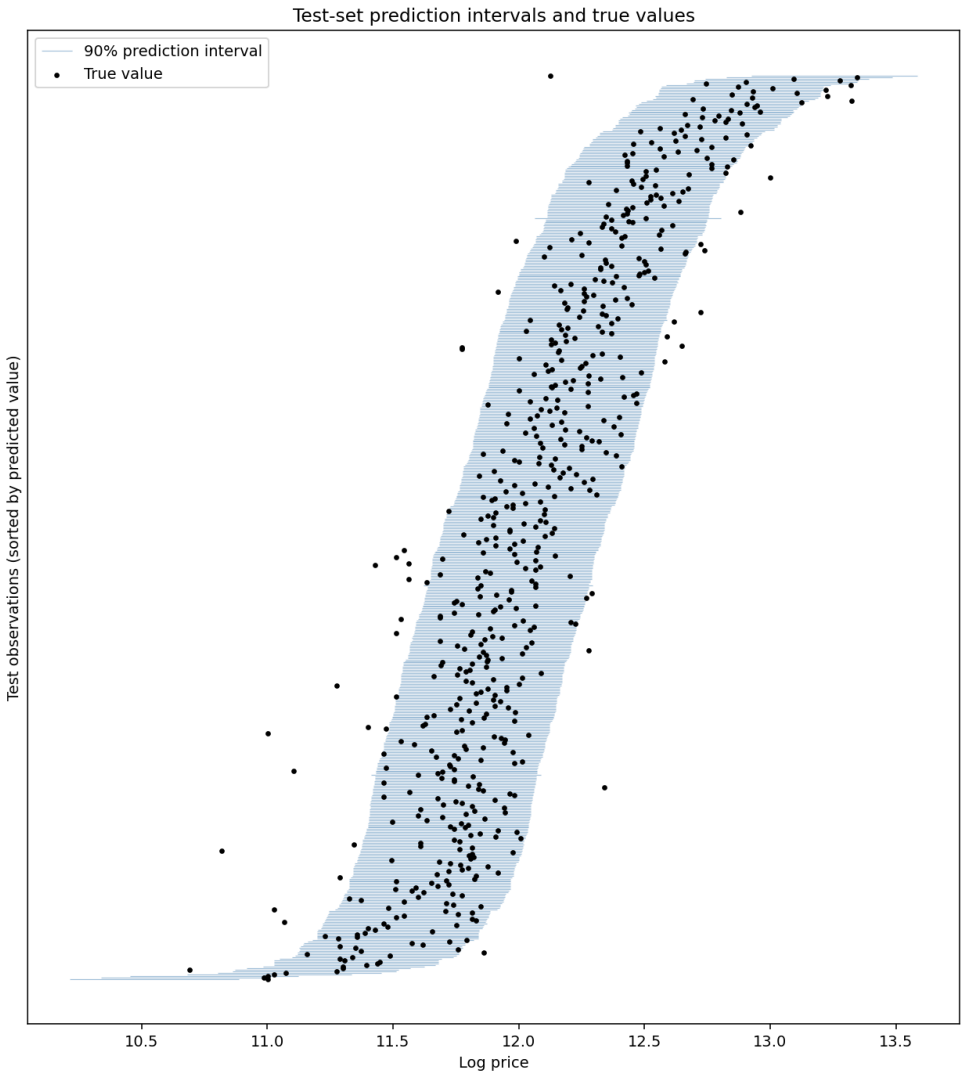
Ames: 90% Prediction Intervals in statsmodels

```
# 90% prediction intervals for individual log prices
prediction = model.get_prediction(test).summary_frame(alpha=0.10)
comparison = pd.DataFrame({
    'actual': np.log(test['price']),
    'predicted': prediction['mean'].to_numpy(),
    'lower': prediction['obs_ci_lower'].to_numpy(),
    'upper': prediction['obs_ci_upper'].to_numpy(),
})
coverage = comparison['actual'].between(comparison['lower'], comparison['upper']).mean()
print(f'Test-set coverage of 90% prediction intervals: {coverage:.1%}')
```

Output: Test-set coverage of 90% prediction intervals: 92.3%

	actual	predicted	lower	upper
1357	11.989160	12.386541	12.065573	12.707509
2367	11.661345	11.820141	11.499755	12.140526
2822	12.188418	12.232951	11.912615	12.553288
2126	11.724806	11.481063	11.160159	11.801967
1544	11.744037	11.775390	11.454938	12.095841

Ames: 90% Prediction Intervals on the Test Set



Each row is a test house, sorted by predicted value. Blue bar = 90% prediction interval; black dot = true log price. Most dots land inside their bars.

MLE for discrete models: Bernoulli distribution

- Let's go back to the binomial example of the coin flipping and formalize it mathematically

$$p[Y = y] = p^y \times (1-p)^{1-y}$$

We know this as the PMF for binomial

$$\begin{aligned} \mathrm{p}(y|p) &= \mathrm{Bin}(y; n, p) \\ &= \binom{n}{y} p^y (1-p)^{n-y} . \end{aligned}$$

Calculate the log likelihood

$$\begin{aligned} \mathrm{LL}(p) &= \log \mathrm{p}(y|p) \\ &= \log \binom{n}{y} + y \log p + (n-y) \log(1-p) . \end{aligned}$$

Take derivative w.r.t. p
Drop the first term without p

$$\frac{\mathrm{dLL}(p)}{\mathrm{d}p} = \frac{y}{p} - \frac{n-y}{1-p}$$

Set the derivative = 0 to find the maximum (p-hat) of the original function (LL)

$$\begin{aligned} \frac{\mathrm{dLL}(p)}{\mathrm{d}\hat{p}} &= 0 \\ 0 &= \frac{y}{\hat{p}} - \frac{n-y}{1-\hat{p}} \\ \frac{n-y}{1-\hat{p}} &= \frac{y}{\hat{p}} \\ (n-y)\,\hat{p} &= y\,(1-\hat{p}) \\ n\,\hat{p} - y\,\hat{p} &= y - y\,\hat{p} \\ n\,\hat{p} &= y \\ \hat{p} &= \frac{y}{n} . \end{aligned}$$

This is just sample proportion

Logit link functions and logistic regression

- For linear regression we “linked” the expected value of y, given x, to the linear predictors with x* b_hat
- For logistic regression, our input is real numbers and our output is 0 or 1
- We need a way to map x*b_hat to that range; let’s add a function to the output (a link function) that transforms y from real numbers to 0-1

The *logit* function,

$$\text{logit}(x) = \log\left(\frac{x}{1-x}\right),$$

maps the range $(0, 1)$ to $(-\infty, \infty)$ and is useful for modeling probabilities. Its inverse function, graphed in Figure 13.1a, maps back to the unit range:

$$\text{logit}^{-1}(x) = \frac{e^x}{1+e^x}. \tag{13.1}$$

- If you set $p_i = x_i\beta$, you can't guarantee that $0 \leq p_i \leq 1$
- Instead, we set

$$p_i = \frac{e^{x_i\beta}}{1+e^{x_i\beta}}$$

Logistic log-likelihood

- Again, we want a max likelihood **estimate of b** to predict **y**
- So we get the log likelihood by substituting this p_i into the Bernoulli likelihood

  - We then substitute this into the Bernoulli likelihood

$$\sum_{i=1}^{n} y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

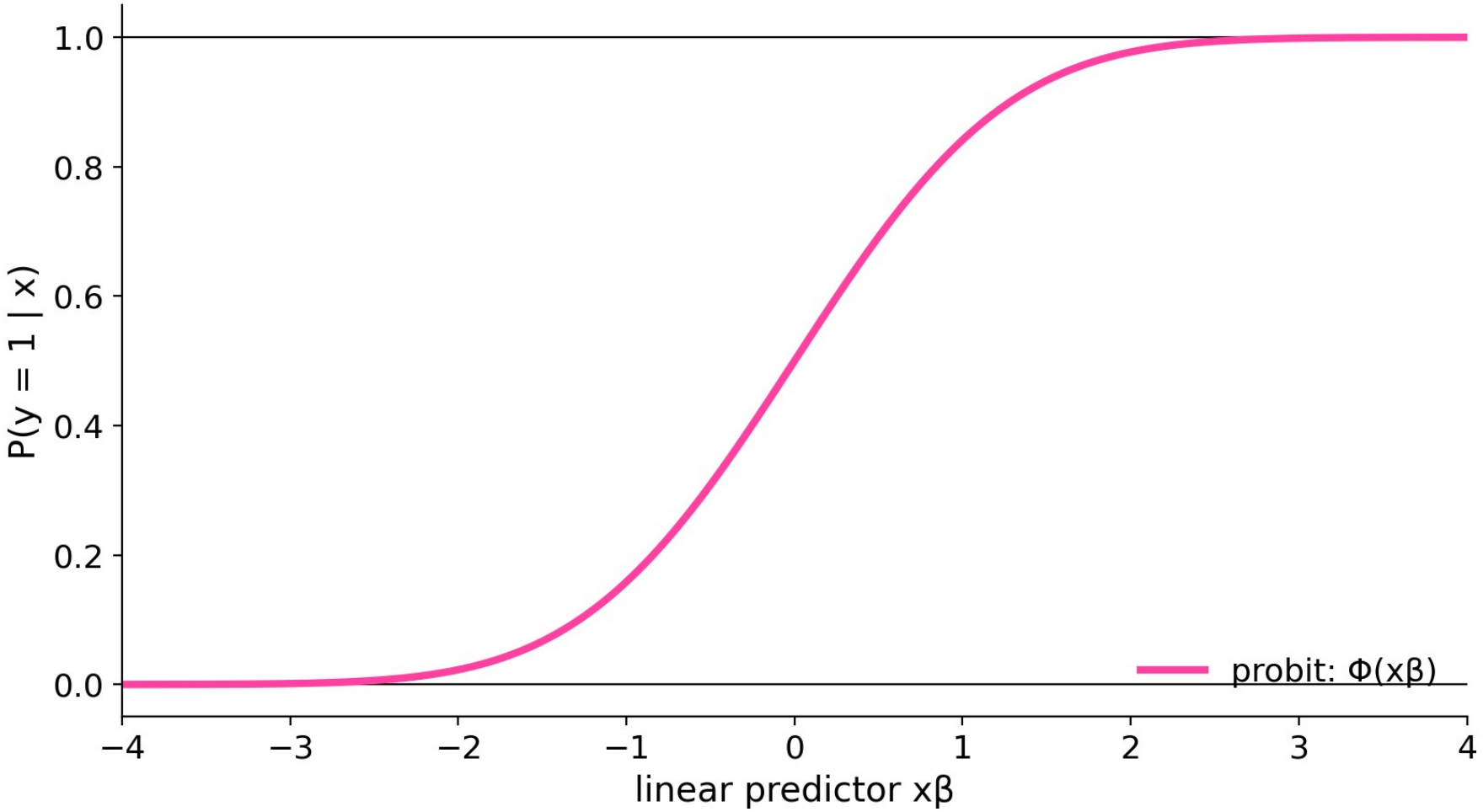
to get

$$\begin{aligned} \ell(\beta) &= \sum_{i=1}^{n} y_i \log\left(\frac{1}{1 + e^{-x_i \cdot \beta}}\right) + (1 - y_i) \log\left(\frac{e^{-x_i \cdot \beta}}{1 + e^{-x_i \cdot \beta}}\right) \\ &= \sum_{i=1}^{n} y_i \log\left(\frac{e^{x_i \cdot \beta}}{1 + e^{x_i \cdot \beta}}\right) + (1 - y_i) \log\left(\frac{1}{1 + e^{x_i \cdot \beta}}\right) \end{aligned}$$

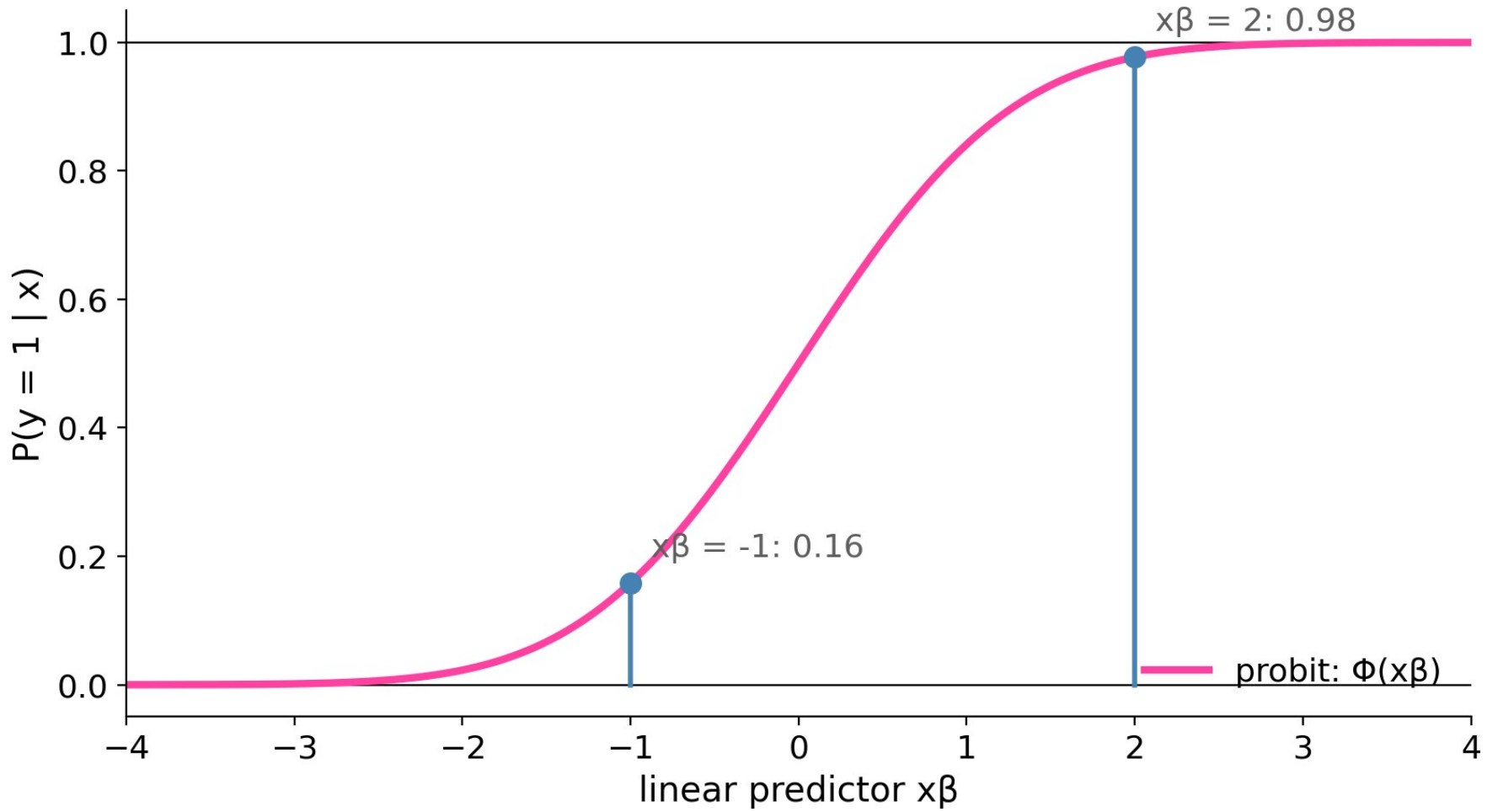
Probit

- We could also have picked a different shape to map real numbers to 0 to 1
- We know of a nice one: the normal CDF
- Probit model: $P(y = 1 \mid x) = \Phi(x\beta)$, where Φ is the normal CDF, so the output is always between 0 and 1
- Likelihood is the Bernoulli one: $\ell(\beta) = \Sigma\ [\ y \log \Phi(x\beta) + (1 - y) \log(1 - \Phi(x\beta))\ ]$

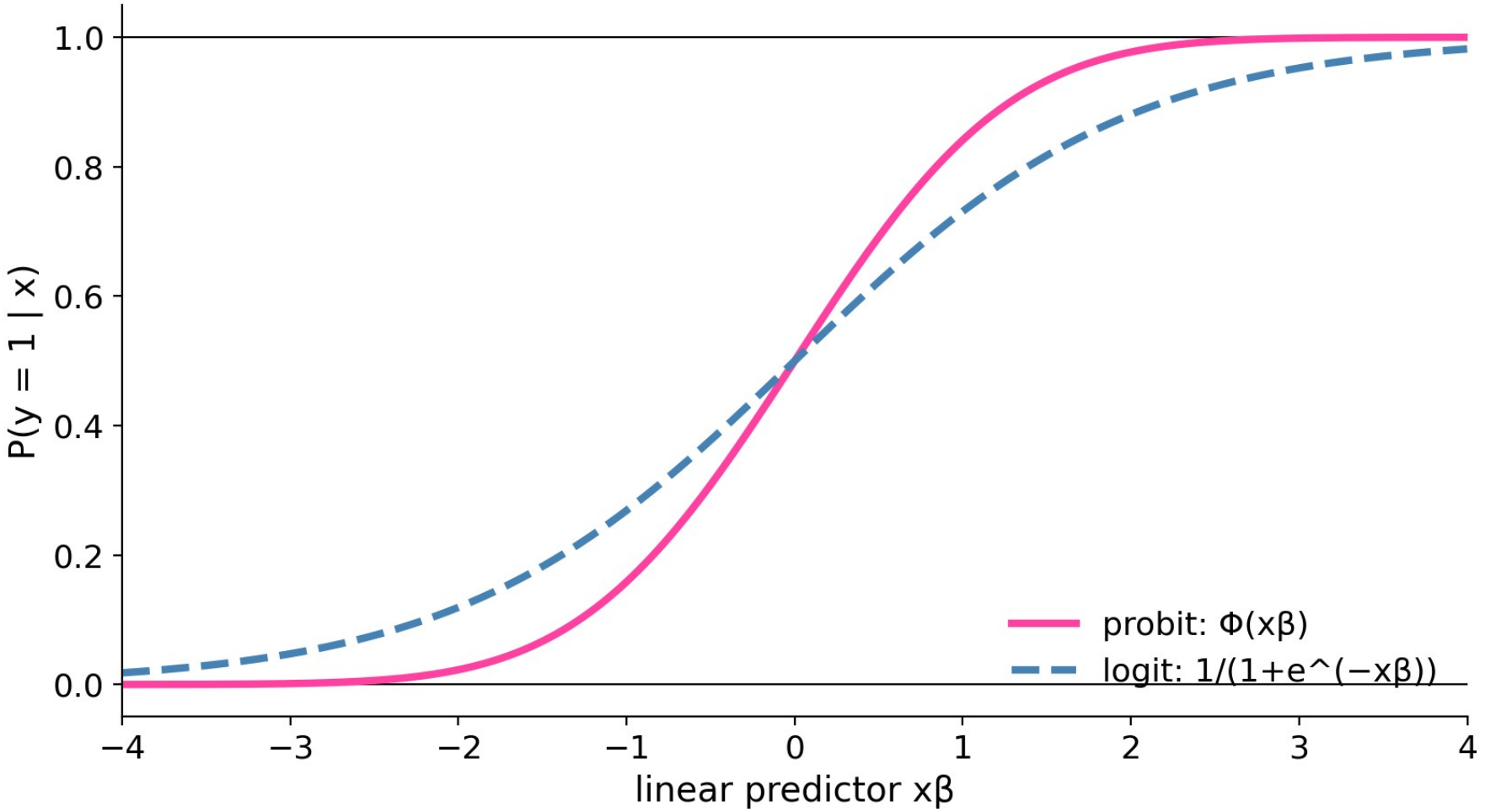
Probit: an S-shaped curve



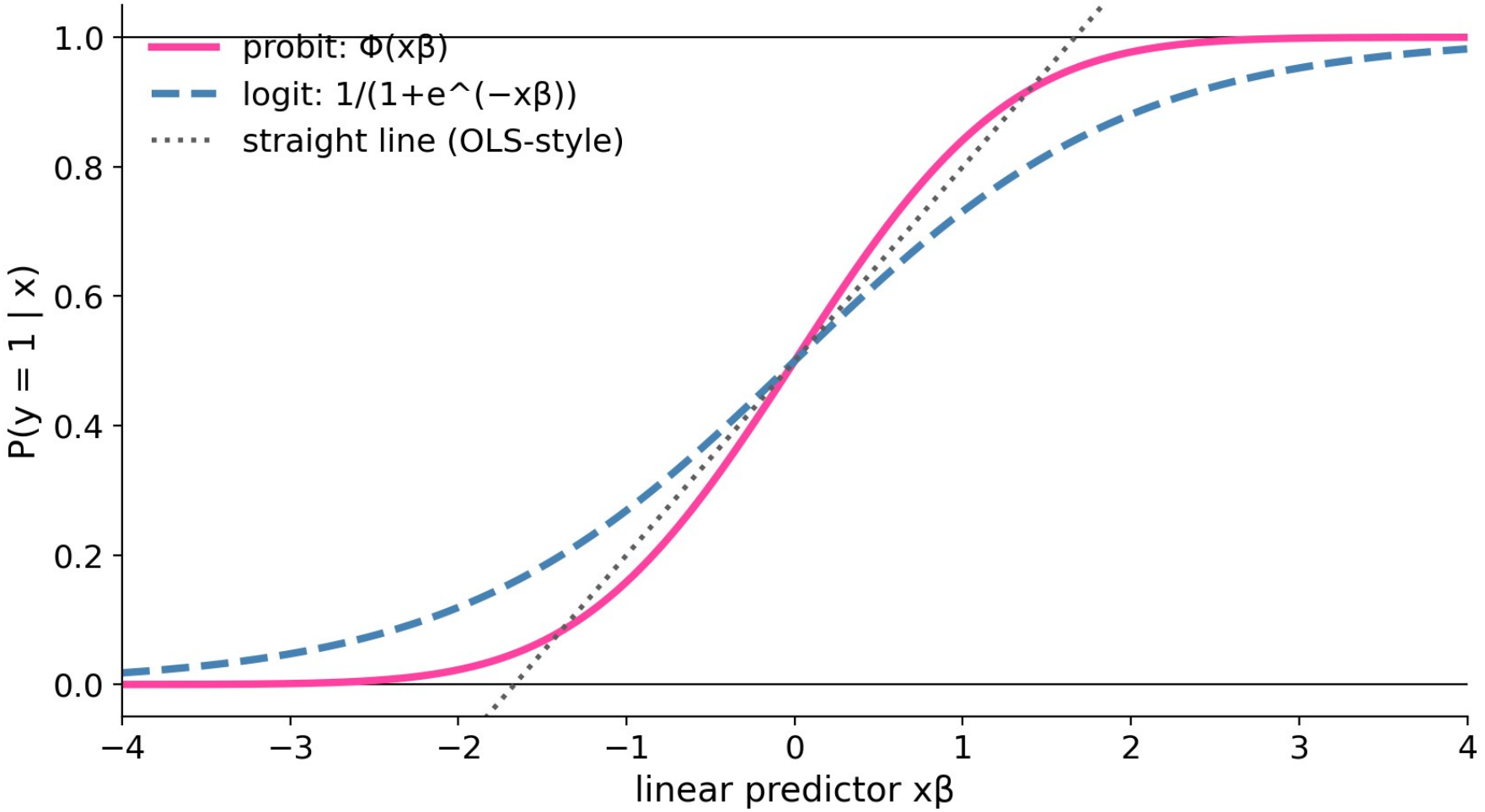
Probit: reading probabilities off the curve



Probit and logit: nearly the same shape



Why not a straight line?



Probit, logit, and OLS

- OLS: $y = x\beta + \varepsilon$ with $\varepsilon \sim N(0, \sigma^2)$; the mean of y is $x\beta$, which can be any real number
- Probit: $P(y = 1 \mid x) = \Phi(x\beta)$, the normal CDF; the mean of y is a probability
- Logit: $P(y = 1 \mid x) = 1 / (1 + e^{(-x\beta)})$, the logistic CDF; also a probability
- Probit and logit have the same structure and usually give very similar predictions
- Use OLS for a continuous outcome; use probit or logit for a 0/1 outcome